2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW) | 978-1-6654-4456-9/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICSTW52544.2021.00054

Flaky Mutants; Another Concern for Mutation Testing

Sten Vercammen
Dept. Computer Science
University of Antwerp
Antwerp, Belgium

Serge Demeyer
Dept. Computer Science
University of Antwerp
Flanders Make, Belgium

Markus Borg
RISE Research Institutes of Sweden
Lund, Sweden

Robbe Claessens
Dept. Computer Science
University of Antwerp
Antwerp, Belgium

Software testing is the dominant method for quality assurance and control in software engineering [1], [2]. Test suites serve as quality gates to safeguard against programming faults. But not every test suite is written equally. We usually gauge its quality using metrics such as code coverage. These assess how much of the code base has been covered. However, they do not tell if the tests actually test and verify the intentions. Mutation testing does this by deliberately injecting faults into the system under test and verifying how many of them the test suite can detect. For every injected fault that is not detected by the test suite, an additional test should be written. In the academic community, mutation testing is acknowledged as the most promising technique for automated assessment of the strength of a test suite [3], [4].

An underlying assumption, rarely mentioned, is that the system under test behaves completely deterministically. Testing for non-determinism in source code is inherently challenging. Testers can only identify non-determinism by confirming the existence of –so called– flaky tests which intermittently pass and fail whilst executing the same code. Non-determinism is impossible to detect from a single execution.

Studies show that flaky tests are generally a substantial and frequent problem [5], [6], [7], [8], [9], [10], [11]. Google stated "Unfortunately, across our entire corpus of tests, we see a continual rate of about 1.5% of all test runs reporting a 'flaky' result". In general, Google noticed that almost 16% of their tests have some level of flakiness associated with them. This means that when a test fails due to flakiness, they lose man-hours, machine capacity, and have costly delays [12]. Furthermore, previous research reported that 13% of all builds fail due to test suites that contain some form of flakiness [13].

Test failures caused by flaky tests are hard to reproduce due to their non-determinism. Consequently, flakiness waste developer time [14]: developers may spend substantial time debugging a failed flaky test only to find out that the failure is not due to the recent changes. Finally, flakiness may even hide bugs [15]: if a flaky test fails frequently, developers tend to ignore its failures and could therefore miss real bugs. Previous studies show that flakiness leads to developers distrusting their test suites – a risky phenomenon in software testing. [15], [16].

As flaky tests are so troublesome, we wonder what this means for mutation testing? Mutation testing increases the chance of flakiness exponentially, as each mutant changes the code. It is thus very likely that some mutants introduce non-determinism. We call these 'flaky' mutants.

Flaky mutants cause the mutation testing score to fluctuate between consecutive mutation analyses as they are only detected intermittently. When a flaky mutant is not detected by the test suite, a developer might try to write a test for it. Afterwards, the mutant may be detected, not due to the added test but due to the flakiness introduced by the flaky mutant. All whilst the actual problem remains unsolved, causing wasted effort and possibly unfruitful tests. Debugging such problems is inevitably time-consuming. Furthermore, these flaky mutants can sometimes act equivalently to the original code, making it an intermittently equivalent mutant and impossible to detect.

When we run a mutation analysis on production code that already contains flakiness, things become even more unpredictable. Introducing a mutant could exaggerate the flakiness, maintain the flakiness, or even fix the flakiness. We currently have no automated way of knowing if a test really detected the mutant or if it fails due to pre-existing flakiness. It is possible that a test fails, causing the mutant to be killed, even though the mutated code was never reached. The test can also fail unrelated to the mutant, even after the mutant was executed.

A parallel can be drawn to traditional flaky tests for which developers lose confidence in the test suite and code base. With flaky mutants, developers might lose confidence in the usefulness of the technique, especially when a developer would intermittently be presented with the same mutant over and over again after he wrote many tests that *detected* it.

Mutants are intended to simulate real faults. When a mutant is not detected by the test suite, it is possible that it is accidentally introduced in the code base. If such a fault is flaky, it might take a while before the fault is found, with potential catastrophic consequences. We found that it is extremely easy to create these flaky mutants and that, anecdotally, every mutant can cause flaky behaviour. As flaky mutants do occur in real life, we should not address them explicitly. However, we cannot write tests that detect flakiness so we should come up with clever techniques. Fortunately, we have an easier challenge to solve than in traditional flaky tests, as we know where the fault is located. Our preliminary findings confirm the presence of flaky mutants in real projects. We also share our initial thoughts on how to tackle the issue.

978-1-6654-4456-9/21/$31.00 ©2021 IEEE
DOI 10.1109/ICSTW52544.2021.00054

Authorized licensed use limited to: BOURNEMOUTH UNIVERSITY. Downloaded on July 06,2021 at 12:11:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Ng, T. Murnane, K. Reed, D. Grant, and T. Chen, "A preliminary survey on software testing practices in australia," in *Software Engineering Conference, 2004. Proceedings. 2004 Australian*. Piscataway, NJ, USA: IEEE Press, 2004, pp. 116–125.

[2] V. Garousi and J. Zhi, "A survey of software testing practices in canada," *Journal of Systems and Software*, vol. 86, no. 5, pp. 1354–1376, 2013.

[3] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, "Mutation testing advances: An analysis and survey," *Advances in Computers*, vol. In Press, no. Corrected Proof, pp. —, Available online 7 May 2018 2019.

[4] Y. Jia and M. Harman, *An Analysis and Survey of the Development of Mutation Testing*. IEEE, 2015. [Online]. Available: https://ieeexplore.ieee.org/document/5487526g

[5] "Avoiding flaky tests," https://testing.googleblog.com/2008/04/tott-avoiding-flakey-tests.html, accessed: 2020-08-14.

[6] "Eradicating non-determinism in tests," https://martinfowler.com/articles/nonDeterminism.html, accessed: 2020-08-14.

[7] "No more flaky tests on the go team," https://www.thoughtworks.com/insights/blog/no-more-flaky-tests-go-team, accessed: 2020-08-14.

[8] A. M. F. Arash Vahabzadeh and A. Mesbah, *An empirical study of bugs in test code*. IEEE, 2015. [Online]. Available: https://ieeexplore.ieee.org/document/7332456

[9] J. B. August Shi and D. Marinov, *Mitigating the effects of flaky tests on mutation testing*. ISSTA, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3293882.3330568

[10] O. L. August Shi, Alex Gyori and D. Marinov, *Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications*. IEEE, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7515461

[11] B. S. Kivanç Muşlu and J. Wuttke, *Finding bugs by isolating unit tests*. ESEC/FSE, 2011. [Online]. Available: https://dl.acm.org/doi/10.1145/2025113.2025202

[12] "Flaky tests at google and how we mitigate them," https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html, accessed: 2020-08-14.

[13] L. I. Adriaan Labuschagne and R. Holmes, *Measuring the cost of regression testing in practice: a study of Java projects using continuous integration*. ESEC/FSE, 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3106237.3106288

[14] F. J. Lacoste, *Killing the Gatekeeper: Introducing a Continuous Integration System*. IEEE, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/5261054

[15] L. E. Qingzhou Luo, Farah Hariri and D. Marinov, *An empirical analysis of flaky tests*. FSE, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/2635868.2635920

[16] "6 tips for writing robust, maintainable unit tests," https://blog.melski.net/tag/unit-tests/, accessed: 2020-08-14.

Authorized licensed use limited to: BOURNEMOUTH UNIVERSITY. Downloaded on July 06,2021 at 12:11:59 UTC from IEEE Xplore. Restrictions apply.